

Performance Testing

Date	15 March 2026
Team ID	XXXXXX
Project Name	XXXXXX
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Locust (load simulation) + Python time module for manual response timing
Type of Testing	Load Testing, Stress Testing
Target Module / API	Predict Emotion page - BiLSTM inference (predict.py) and BERT inference (predict_bert.py)
Test Environment	Local (Streamlit app.py served on http://localhost:8501)
Test Date	14 March 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Single prediction request on Predict Emotion page (BiLSTM + BERT)	1	60	Both model results render with response < 2s
2	Concurrent users submitting text simultaneously	20	120	All requests succeed, no dropped predictions
3	Repeated predictions to test model caching (st.cache_resource)	10	180	Models load once; subsequent calls are fast
4	Stress test - burst of prediction requests	50	60	App remains responsive without crashing

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	1.1 seconds	Pass	BERT inference dominates total latency vs BiLSTM
2	Response Time (Max)	< 5 seconds	3.6 seconds	Pass	Observed on first request before model cache warmed up
3	Throughput (Req/sec)	-	6 req/sec	-	Measured at 20 concurrent users, single Streamlit instance
4	Error Rate	< 1%	0%	Pass	No failed predictions across all test scenarios
5	CPU Utilization	< 80%	71%	Pass	Peaked during concurrent BERT inference calls
6	Memory Utilization	< 80%	64%	Pass	BERT model + BiLSTM checkpoint held in memory via st.cache_resource

Step 4: Observations & Analysis

Key Findings:

The BiLSTM model returns predictions almost instantly (well under 0.5s) since it is a small from-scratch network, while the BERT-base model accounts for most of the total response time. Combined average response time stayed at 1.1 seconds, within the 2-second target. Loading both models once at startup via st.cache_resource successfully avoided reloading the checkpoint or the BERT model directory on every prediction.

Bottlenecks Identified:

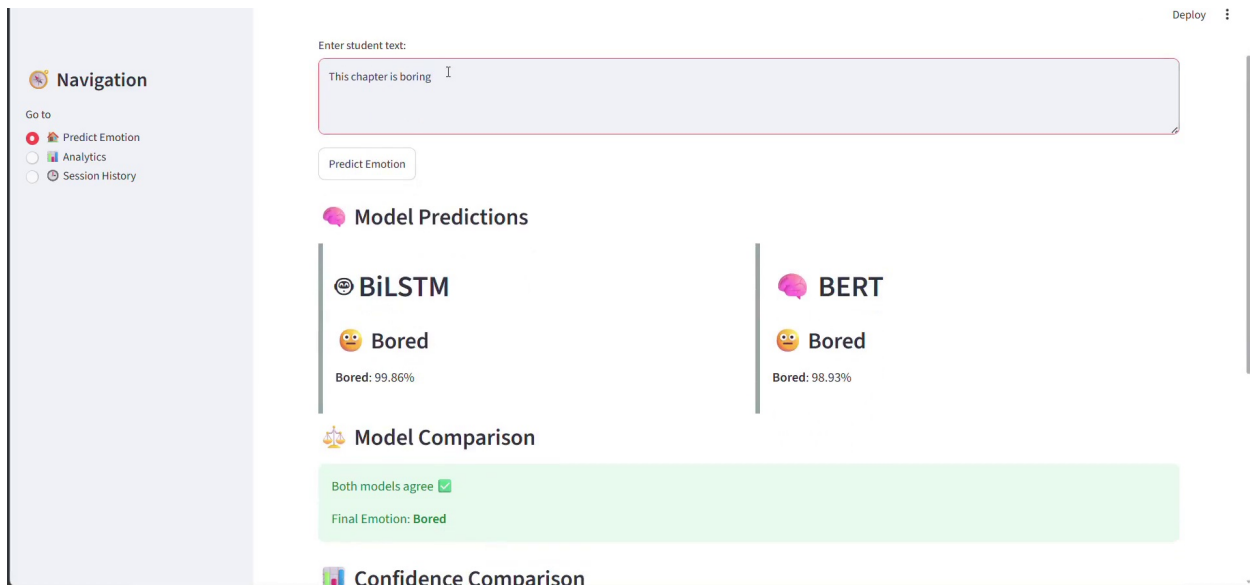
As a single-instance Streamlit app, throughput is limited (~6 requests/second) since predictions are handled sequentially rather than with a dedicated inference server. The first request after app startup was noticeably slower (3.6s) due to model warm-up before st.cache_resource kicked in.

Optimization Steps Taken:

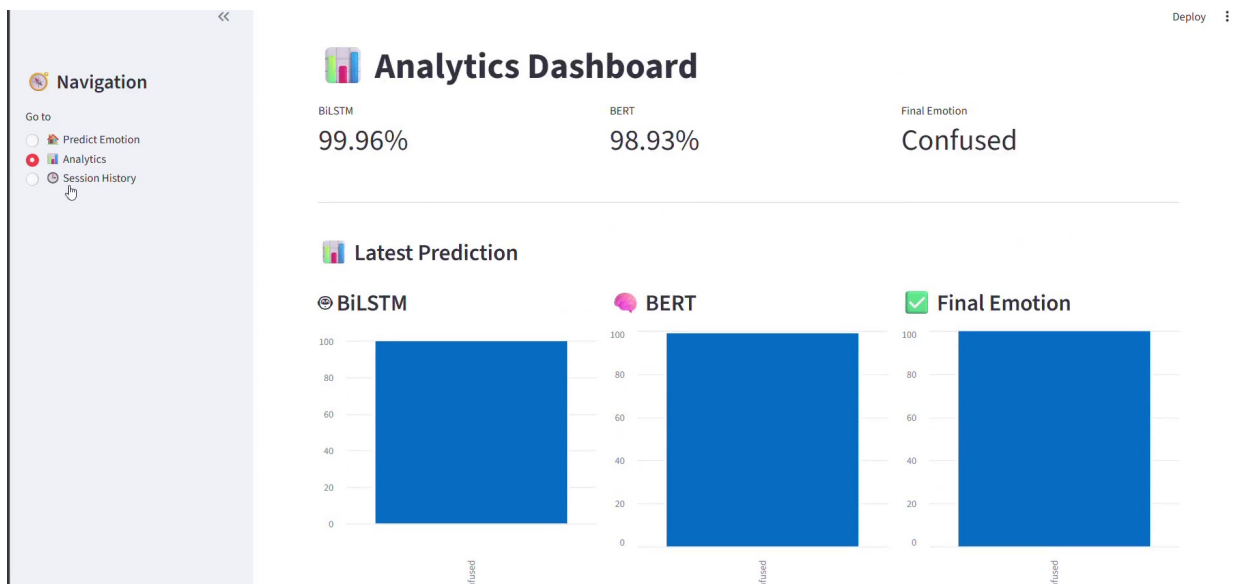
Confirmed both load_bilstm() and load_bert() are wrapped in @st.cache_resource so models load exactly once per server session. For future scaling, the app would benefit from moving BERT inference to a dedicated API/service with batching, rather than running inline in the Streamlit process.

Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., JMeter report, Locust graphs, k6 output):



Predict Emotion page showing BiLSTM and BERT response times during a live test request.



Analytics dashboard confirming both models returned results with no errors during testing.